

内存管理：分页、分段、段页式管理完整笔记

内存管理：分页、分段、段页式管理完整笔记

目标：把“页、页面、页框、页帧、块、页号、页框号、物理块号、地址变换”等基本概念一次讲清楚，并能看懂 408/操作系统题目中的地址结构与转换过程。

0. 先建立整体图景

程序运行时看到的是“逻辑地址空间”，真实内存里放的是“物理地址空间”。内存管理要解决的核心问题是：

- 1. 程序以为自己从地址 0 开始连续运行，但物理内存不一定连续。
- 2. 多个进程同时运行，必须隔离和保护。
- 3. 主存不够时，要能把暂时不用的内容放到外存，需要时再调入。
- 4. CPU 每次访存给出的逻辑地址，必须快速变成物理地址。

因此出现了三类典型管理方式：

管理方式	逻辑空间怎么切	物理内存怎么分配	地址形式	核心优点	核心问题
分页	按固定大小切成页	按同样大小切成页框	页号 + 页内偏移	消除外部碎片，方便离散分配	有内部碎片，页表可能大
分段	按程序逻辑模块切成段	段大小可变，通常连续分配	段号 + 段内偏移	符合程序逻辑，方便共享和保护	有外部碎片
段页式	先按段，再把每段分页	页框离散分配	段号 + 页号 + 页内偏移	兼顾逻辑保护与离散分配	地址变换更复杂，表更多

一句话记忆：分页是“物理友好”，分段是“程序员/逻辑友好”，段页式是“先逻辑分段，再物理分页”。

1. 基本概念总表

1.1 逻辑地址与物理地址

逻辑地址也叫虚拟地址，是 CPU 执行程序时产生的地址。程序中的指针、数组下标、函数入口地址，看到的都是逻辑地址。

物理地址是主存条中真实的地址。CPU 最终必须拿到物理地址，才能真正访问内存单元。

地址变换就是：

逻辑地址 → 地址变换机构/MMU → 物理地址

这里的 MMU 是 Memory Management Unit，即内存管理单元，通常由硬件完成核心转换，操作系统负责维护页表、段表等数据结构。

1.2 页、页面、页框、页帧、块、物理块

这些词最容易混。可以按“虚拟空间”和“物理空间”分开看。

名词	所属空间	含义	大小是否固定	常见别名
页	逻辑地址空间	进程逻辑空间被切成的固定大小单位	固定	页面、虚页、虚拟页
页面	逻辑地址空间	通常就是“页”的另一种说法	固定	页
页框	物理内存	物理内存被切成的固定大小单位	固定	页帧、物理页框
页帧	物理内存	通常就是“页框”的另一种说法	固定	frame、page frame
物理块	物理内存语境	在分页管理中常指页框	固定	物理页框、内存块
块	语境相关	可能指磁盘块，也可能指内存块	看语境	block

注意：**页和页框大小必须相等**。因为一个逻辑页面调入主存时，要刚好放进一个物理页框。

所以在分页题目中，常可认为：

$$\text{页大小} = \text{页框大小} = \text{物理块大小}$$

但是“块”这个字要看上下文：

- 在文件系统里，“块”常指磁盘块。
- 在分页存储里，“块”常指物理内存块，也就是页框/页帧。
- 在 Cache 里，“块”可能指 Cache 块或主存块。

1.3 页号、页框号、物理块号

设一个进程的逻辑空间被切成很多页：第 0 页、第 1 页、第 2 页……这些编号就是**页号**。

设物理内存被切成很多页框：第 0 号页框、第 1 号页框、第 2 号页框……这些编号就是**页框号**。

很多教材把页框号也叫**物理块号**。在分页地址变换中：

逻辑地址中的编号	页表查到的编号	物理地址中的编号
页号	页框号/物理块号	页框号/物理块号

核心关系是：

$$\text{页号} \xrightarrow{\text{查页表}} \text{页框号}$$

偏移量不变：

$$\text{页内偏移} = \text{页框内偏移}$$

2. 分页管理

2.1 分页的基本思想

分页管理把进程的逻辑地址空间分成大小相等的页，把物理内存分成同样大小的页框。进程的每一页可以装入任意一个空闲页框，不要求连续。

例如，一个进程有 4 页：

逻辑页：	页0	页1	页2	页3
物理页框：	框5	框2	框9	框1

这表示：

逻辑页	实际装入的物理页框
页 0	页框 5
页 1	页框 2
页 2	页框 9
页 3	页框 1

进程以为页 0、页 1、页 2、页 3 是连续的，但实际在物理内存中可以分散存放。

2.2 页大小

页大小通常是 2 的整数次幂，例如 4 KB、8 KB、16 KB。这样地址拆分非常方便。

如果页大小为：

$$L = 2^n$$

则页内偏移需要 n 位。

如果逻辑地址长度为 m 位，则：

$$\begin{aligned} \text{页内偏移位数} &= n \\ \text{页号位数} &= m - n \\ \text{逻辑页数} &= 2^{m-n} \end{aligned}$$

例如页大小是 4 KB：

$$4\text{KB} = 4096\text{B} = 2^{12}\text{B}$$

所以页内偏移占 12 位。

2.3 分页逻辑地址结构

分页系统中的逻辑地址通常分成两部分：

逻辑地址 A： 页号 p 页内偏移 d

其中：

- p ：页号，表示访问逻辑地址空间中的第几页。
- d ：页内偏移，表示在该页内部距离页首多少字节。

如果页大小为 L ，逻辑地址为 A ，则：

$$p = \left\lfloor \frac{A}{L} \right\rfloor$$

$$d = A \bmod L$$

2.4 页表

页表是操作系统为每个进程维护的数据结构，用来记录“逻辑页号到物理页框号”的映射。

页表最核心字段是：

页号	页框号
0	5
1	2
2	9
3	1

页表项通常不止页框号，还可能包括：

字段	作用
页框号/物理块号	该页在物理内存中的位置
有效位/存在位	该页是否已经在内存中
访问权限位	可读、可写、可执行等
修改位/脏位	该页是否被写过，换出时是否要写回外存
访问位/引用位	页面置换算法使用，如 CLOCK、LRU 近似
外存地址	页面不在内存时，它在外存的位置

2.5 分页地址变换过程

假设逻辑地址为 A ，页大小为 L ，页表中页号 p 对应的页框号为 f 。

地址变换过程：

1. 根据逻辑地址计算页号和页内偏移：

$$p = \left\lfloor \frac{A}{L} \right\rfloor$$

$$d = A \bmod L$$

2. 查页表，得到页号 p 对应的页框号 f 。

3. 拼接页框号和页内偏移，得到物理地址：

$$E = f \times L + d$$

也可以理解为：

逻辑地址：| 页号 p | 页内偏移 d |
 ↓ 查页表

物理地址：| 页框号 f | 页内偏移 d |

重点：页内偏移不变，只把页号替换成页框号。

2.6 分页地址变换例题

已知：

- 逻辑地址为 $A = 0x12345$ 。
- 页大小为 4 KB，即 $L = 2^{12}$ 。
- 页号 $0x12$ 对应的页框号为 $0x3A$ 。

因为页大小为 2^{12} ，所以低 12 位是页内偏移：

逻辑地址 $0x12345 =$ | 页号 $0x12$ | 页内偏移 $0x345$ |

查页表得到：

$$0x12 \longrightarrow 0x3A$$

所以物理地址为：

物理地址 $=$ | 页框号 $0x3A$ | 页内偏移 $0x345$ | $= 0x3A345$

用公式写就是：

$$E = 0x3A \times 0x1000 + 0x345 = 0x3A345$$

2.7 页表基址寄存器和页表长度寄存器

为了找到页表，系统通常需要两个寄存器：

寄存器	作用
页表基址寄存器	保存当前进程页表在内存中的起始地址
页表长度寄存器	保存当前进程页表长度，用于检查页号是否越界

地址变换时，硬件先检查页号是否超过页表长度。若页号非法，则发生地址越界异常。

2.8 TLB：快表

如果每次访存都要先访问页表，再访问真正的数据，那么一次逻辑访存可能变成两次物理访存：

1. 访问页表，得到页框号。
2. 访问目标物理地址，取数据或写数据。

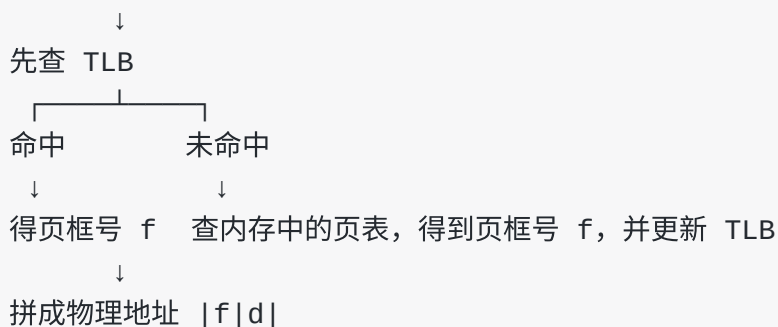
为了加速，CPU 使用 TLB，也叫快表。TLB 是页表项的高速缓存，保存最近使用过的页号到页框号映射。

地址变换流程变成：

CPU 产生逻辑地址

↓

拆出页号 p 和页内偏移 d



如果 TLB 命中, 则不需要访问内存中的页表, 速度会快很多。

2.9 缺页中断

如果页表项的有效位显示该页不在内存中, 就会发生**缺页中断**。

缺页中断的大致流程:

1. CPU 访问某个逻辑页。
2. 查页表发现该页不在内存。
3. 硬件产生缺页中断, 转入操作系统。
4. 操作系统在外存中找到该页。
5. 找一个空闲页框; 若没有空闲页框, 则选择一个页面换出。
6. 把需要的页面调入页框。
7. 修改页表项, 把有效位设为有效, 并填入页框号。
8. 重新执行被中断的指令。

缺页中断属于异常中的一种, 因为它由正在执行的指令引发, 并且处理后通常可以重新执行该指令。

2.10 分页的碎片问题

分页不会产生外部碎片, 因为任意空闲页框都能分配给任意逻辑页。

但是分页会产生内部碎片。原因是最后一页往往用不满。

例如进程大小为 18 KB, 页大小为 4 KB:

$$18\text{KB} = 4\text{KB} \times 4 + 2\text{KB}$$

它需要 5 页, 其中最后一页只用 2 KB, 剩下 2 KB 浪费在该页内部。这就是内部碎片。

2.11 多级页表

如果逻辑地址空间很大, 单级页表也会很大。多级页表把页表本身再分页, 只在需要时分配页表页。

例如二级页表地址结构:

逻辑地址: $| \text{一级页号 } p_1 | \text{ 二级页号 } p_2 | \text{ 页内偏移 } d |$

地址变换:

p1 查一级页表 → 找到二级页表
p2 查二级页表 → 找到页框号 f
f 与 d 拼接 → 物理地址

多级页表节省页表空间，但可能增加访存次数，所以仍然依赖 TLB 提速。

3. 分段管理

3.1 分段的基本思想

分页按固定大小机械切分，而分段按程序的逻辑结构切分。

一个程序可以分成：

- 主程序段。
- 子程序段。
- 函数库段。
- 全局数据段。
- 栈段。
- 堆段。

每个段有自己的长度和含义。段大小可以不同，并且段长通常会随程序逻辑变化。

3.2 分段逻辑地址结构

分段系统中的逻辑地址通常写成：

逻辑地址：| 段号 s | 段内偏移 d |

或者表示为二元组：

(s, d)

其中：

- s ：段号，表示访问哪个逻辑段。
- d ：段内偏移，表示访问该段内部距离段首多少字节的位置。

3.3 段表

段表记录每个段在物理内存中的位置和长度。

典型段表项包括：

段号	段长/界限	基址	权限
0	20 KB	100 KB	读/执行
1	12 KB	300 KB	读/写
2	8 KB	500 KB	读/写

其中：

- 基址：该段装入物理内存后的起始地址。
- 段长/界限：该段的长度，用于判断段内偏移是否越界。
- 权限：是否允许读、写、执行。

3.4 分段地址变换过程

逻辑地址为 (s, d) 。段表中第 s 项的基址为 b_s ，段长为 l_s 。

地址变换过程：

1. 检查段号 s 是否超过段表长度。
2. 查段表得到段长 l_s 和基址 b_s 。
3. 检查段内偏移是否越界：

$$d < l_s$$

4. 若合法，则物理地址为：

$$E = b_s + d$$

如果 $d \geq l_s$ ，说明访问超过了该段长度，产生越界异常或段错误。

3.5 分段地址变换例题

已知段表：

段号	段长	基址
0	1000	4000
1	500	9000
2	800	12000

访问逻辑地址 $(1, 320)$ 。

因为段号为 1，查段表得：

$$l_1 = 500$$

$$b_1 = 9000$$

检查：

$$320 < 500$$

合法，所以物理地址为：

$$E = 9000 + 320 = 9320$$

若访问 $(1, 700)$ ，则：

$$700 \geq 500$$

非法，发生越界。

3.6 分段的优缺点

分段的优点：

1. 符合程序逻辑，便于按函数、模块、数据结构组织内存。
2. 便于共享。例如多个进程共享同一个只读代码段。
3. 便于保护。例如代码段只读可执行，数据段可读写，栈段不可执行。
4. 段长可以按逻辑模块自然增长或变化。

分段的缺点：

1. 每段通常需要连续的物理内存空间。
2. 会产生外部碎片。
3. 需要进行内存紧凑时，移动段的代价较大。

3.7 分段的碎片问题

分段的段长可变，物理内存中会出现很多零散小空闲区。如果某个新段需要较大连续空间，即使总空闲空间足够，也可能因为不连续而无法分配。这就是外部碎片。

例如空闲区分别为 2 KB、3 KB、4 KB，总共 9 KB，但新段需要 6 KB 连续空间，则无法直接装入。

4. 段页式管理

4.1 为什么需要段页式

分页解决了离散分配问题，但不体现程序逻辑。

分段体现程序逻辑，但要求每个段连续存放，容易产生外部碎片。

段页式把两者结合起来：

1. 先把程序按逻辑分成多个段。
2. 再把每个段内部按固定大小分页。
3. 每个页面装入任意物理页框。

因此段页式的本质是：

程序逻辑空间：先分段
每个段内部：再分页
物理内存：按页框离散分配

4.2 段页式逻辑地址结构

段页式地址通常分为三部分：

逻辑地址：| 段号 s | 段内页号 p | 页内偏移 d |

也可以写成三元组：

(s, p, d)

其中：

- s : 段号，表示访问哪个段。
- p : 段内页号，表示访问该段中的第几页。
- d : 页内偏移，表示访问该页内部的哪个位置。

4.3 段表和页表

段页式中，每个进程有一张段表，每个段又有自己的页表。

段表项通常保存：

字段	含义
页表始址	该段对应页表的起始地址
页表长度	该段有多少页
段权限	读、写、执行、共享等

段内页表项保存：

字段	含义
页框号	该段内某页实际装入的物理页框
有效位	是否在内存中
修改位	是否被写过
权限位	页面级访问权限

4.4 段页式地址变换过程

逻辑地址为 (s, p, d) 。

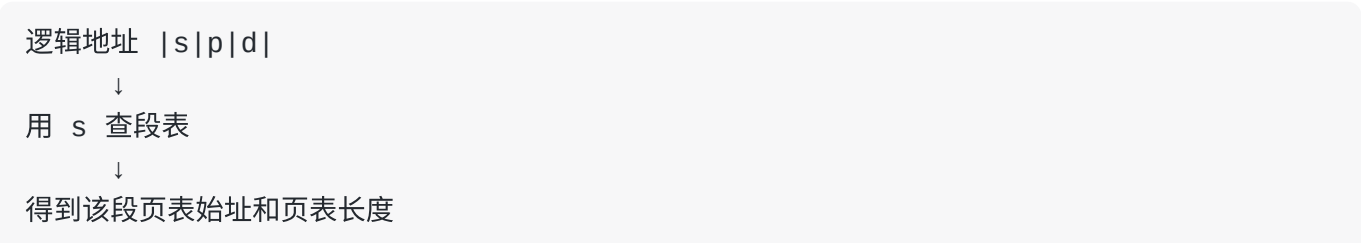
地址变换过程：

1. 根据段号 s 查段表。
2. 检查段号是否合法。
3. 由段表项得到该段页表的起始地址和页表长度。
4. 检查段内页号 p 是否越界。
5. 根据 p 查该段的页表，得到页框号 f 。
6. 物理地址为：

$$E = f \times L + d$$

其中 L 是页大小。

流程图可以写成：



↓
用 p 查该段页表
↓
得到页框号 f
↓
物理地址 $|f|d|$

段页式中，段号和页号都会变换，最后真正进入物理地址的是页框号和页内偏移。

4.5 段页式地址变换例题

已知页大小为 1 KB，某进程段表如下：

段号	页表长度	页表说明
0	3	段 0 有 3 页
1	2	段 1 有 2 页

段 1 的页表如下：

段内页号	页框号
0	8
1	12

访问逻辑地址 $(1, 1, 200)$ 。

步骤：

1. 段号 $s = 1$ ，查段表，段 1 存在。
2. 段 1 页表长度为 2，合法页号为 0 和 1。
3. 段内页号 $p = 1$ ，合法。
4. 查段 1 页表，页号 1 对应页框号 12。
5. 页大小 $L = 1024$ ，页内偏移 $d = 200$ 。

所以物理地址为：

$$E = 12 \times 1024 + 200 = 12488$$

5. 三种方式的对比

5.1 地址结构对比

管理方式	地址结构	地址变换核心
分页	p, d	页号 p 查页表得到页框号 f ，再拼 d
分段	s, d	段号 s 查段表得到基址 b_s ，再加 d
段页式	s, p, d	段号找段页表，页号找页框号，再拼 d

5.2 碎片对比

管理方式	内部碎片	外部碎片	原因
分页	有	无	页固定大小，最后一页可能用不满；页框可离散分配
分段	无或较少	有	段按实际长度分配，但要求连续空间
段页式	有	无	段内分页，所以最后一页可能用不满；物理页框离散分配

更准确地说，分段按实际段长申请，通常不讨论固定页大小带来的内部碎片；但内存分配器若有对齐粒度，也可能出现少量内部浪费。408 语境下通常记为：**分段有外部碎片，分页有内部碎片，段页式有内部碎片但无外部碎片。**

5.3 共享与保护对比

管理方式	共享	保护
分页	可以共享页面，但逻辑意义不直观	页面级保护
分段	很自然，代码段/数据段可独立共享	段级保护，符合逻辑模块
段页式	以段为逻辑单位共享，以页为物理单位存放	段级 + 页级保护都可以做

5.4 地址变换速度对比

管理方式	不考虑 TLB 的访存特点
分页	先访问页表，再访问目标数据
分段	先访问段表，再访问目标数据
段页式	先访问段表，再访问页表，再访问目标数据

段页式表面上访存次数更多，所以更依赖 TLB。现代系统一般会用 TLB 缓存最终的虚拟页到物理页框映射，从而避免每次都完整走多级结构。

6. 地址变换中的常见寄存器和数据结构

6.1 页表基址寄存器

页表基址寄存器保存当前进程页表在内存中的起始地址。进程切换时，操作系统会切换这个寄存器，使 CPU 使用新进程的页表。

6.2 段表基址寄存器

段表基址寄存器保存当前进程段表在内存中的起始地址。分段和段页式都需要它。

6.3 长度寄存器

页表长度寄存器或段表长度寄存器用于越界检查。

例如分页中，若页号 p 满足：

$$p \geq \text{页表长度}$$

则页号越界。
分段中，若段号 s 满足：

$$s \geq \text{段表长度}$$

则段号越界。

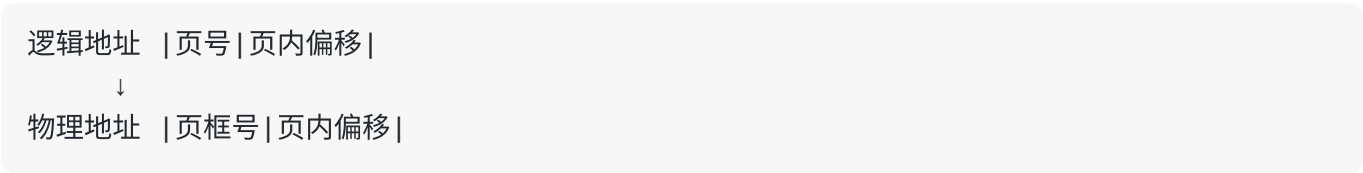
6.4 页表项、段表项、段页式表项

表项	必有字段	典型作用
页表项	页框号、有效位	页号到页框号映射
段表项	基址、段长	段号到物理连续区间映射
段页式段表项	页表始址、页表长度	段号到该段页表的映射
段页式页表项	页框号、有效位	段内页号到页框号映射

7. 典型易混点

7.1 页号是不是物理地址的一部分？

不是。页号属于逻辑地址。物理地址中对应位置是页框号。
分页变换本质是：



7.2 页框号和物理块号是不是一回事？

在分页存储管理语境下，基本可以认为是一回事。教材说“页号对应物理块号”，就是页号通过页表查到页框号。
但如果题目讨论磁盘、文件系统、Cache，则“块”可能不是页框。一定要看题目语境。

7.3 为什么页内偏移不变？

因为页和页框大小相等。逻辑页中的第 d 个字节，装入物理页框后仍然是该页框中的第 d 个字节。
所以只需要改变“在哪一页框”，不需要改变“页内第几个位置”。

7.4 页表保存在哪里？

页表通常保存在内存中，由页表基址寄存器指向。TLB 只是页表项的缓存，不是完整页表。

7.5 分页为什么没有外部碎片？

因为物理内存被划成固定大小页框，只要有空闲页框，就能装入任意页面，不要求连续。

7.6 分段为什么有外部碎片？

因为每段长度不同，且通常要求段在物理内存中连续存放。多次分配和释放后，空闲空间会变得零散。

7.7 段页式有没有外部碎片？

通常认为没有外部碎片。因为段内部已经分页，物理内存按页框分配，不要求整个段连续。但它仍可能有内部碎片，因为每个段的最后一页可能用不满。

7.8 分页和分段谁对用户可见？

一般说：

- 分页对用户透明，程序员通常感受不到页面划分。
- 分段对用户可见或更接近程序员视角，因为段对应代码、数据、栈等逻辑模块。

7.9 逻辑地址空间和物理地址空间大小必须一样吗？

不必须。虚拟存储系统允许逻辑地址空间大于物理内存。只要当前需要的页面在内存中即可，不在内存的页面可以放在外存。

8. 408 常见计算套路

8.1 已知页大小和逻辑地址，求页号和页内偏移

页大小为 L ，逻辑地址为 A ：

$$p = \left\lfloor \frac{A}{L} \right\rfloor$$
$$d = A \bmod L$$

如果 $L = 2^n$ ，则二进制地址低 n 位是页内偏移，高位是页号。

8.2 已知页表，求物理地址

步骤固定：

逻辑地址 A
→ 拆成页号 p 和页内偏移 d
→ 查页表， p 对应页框号 f
→ 物理地址 $E = f \times \text{页大小} + d$

8.3 已知逻辑地址位数和页大小，求页表项数量

逻辑地址为 m 位，页大小为 2^n 字节，则页号位数为 $m - n$ ，逻辑页数为：

$$2^{m-n}$$

单级页表项数量就是逻辑页数。

8.4 已知物理地址位数和页大小，求页框数

物理地址为 k 位，页大小为 2^n 字节，则页框号位数为 $k - n$ ，物理页框数为：

$$2^{k-n}$$

8.5 已知页表项大小，求页表占用空间

若逻辑页数为 N ，每个页表项大小为 e 字节，则页表大小为：

$$N \times e$$

如果要换算成页表页数量，再除以页大小并向上取整：

$$\left\lceil \frac{N \times e}{L} \right\rceil$$

8.6 分段地址合法性判断

逻辑地址为 (s, d) 。

1. 判断段号 s 是否小于段表长度。
2. 判断段内偏移 d 是否小于该段段长。
3. 合法则物理地址为 $b_s + d$ 。

8.7 段页式地址合法性判断

逻辑地址为 (s, p, d) 。

1. 判断段号 s 是否合法。
2. 查段表，得到该段页表长度。
3. 判断段内页号 p 是否小于该段页表长度。
4. 查该段页表，得到页框号 f 。
5. 物理地址为 $fL + d$ 。

9. 一个统一记忆模型

把内存管理看成“查表替换前缀，偏移保持局部意义”。

分页：

```
| 页号 p | 页内偏移 d |
      ↓ 查页表
| 框号 f | 页内偏移 d |
```

分段：

```
| 段号 s | 段内偏移 d |
      ↓ 查段表
物理地址 = 段基址 b_s + d
```

段页式：

```
| 段号 s | 段内页号 p | 页内偏移 d |
      ↓ 查段表
```

找到该段页表

↓ 查页表

| 框号 f | 页内偏移 d |

最核心的三条公式：

$$\text{分页: } E = fL + d$$

$$\text{分段: } E = b_s + d$$

$$\text{段页式: } E = fL + d$$

其中分页和段页式最后都是“页框号 + 页内偏移”，分段最后是“段基址 + 段内偏移”。

10. 一页速记

1. 页/页面：逻辑空间单位。
2. 页框/页帧：物理内存单位。
3. 物理块号：分页语境下通常就是页框号。
4. 页号在逻辑地址中，页框号在物理地址中。
5. 页大小等于页框大小，所以页内偏移不变。
6. 分页地址： (p, d) ，查页表，得到 (f, d) 。
7. 分段地址： (s, d) ，查段表，得到 $b_s + d$ 。
8. 段页式地址： (s, p, d) ，先查段表，再查页表，得到 (f, d) 。
9. 分页：有内部碎片，无外部碎片。
10. 分段：有外部碎片，便于共享和保护。
11. 段页式：保留段的逻辑优点，同时用分页消除外部碎片。
12. TLB 是页表项缓存，用来减少地址变换带来的额外访存。
13. 缺页中断说明访问的页不在内存，操作系统需要把它从外存调入。
14. 做题时先看页大小，再拆地址；低位通常是偏移，高位通常是编号。